Univariate Data Analysis & Visualization

Wook-Shin Han

Spring 2026

What to Use When (Quick Guide)

- **Simple shape overview:** Histogram (tune `bins`; normalize if sizes differ).
- **Bin-free, exact cumulative view:** ECDF (excellent for unequal n).
- **Smooth shape & modes:** KDE (choose bandwidth; reflect/transform if bounded).
- **Compact group comparison:** Box; with large n, Boxen.
- **Show observations:** Rug/Strip; overlay on Box/Violin for validation.
- **Normality / tail checks:** Q–Q plot.
- **Heavy right tails or non-negativity:** log or related transforms.

Real Datasets & Setup

Datasets (from scikit-learn):

- **Iris** (150 flowers): `sepal length (cm)`, `sepal width (cm)`, `petal length (cm)`, `petal width (cm)`, `target` (`0,1,2` species).
- **Wine** (178 samples): physicochemical features (e.g., `alcohol`, `flavanoids`), `target` (classes `0,1,2`).
- **Diabetes** (442 patients): standardized features (e.g., `bmi`), `target` (disease progression).

We assume a DataFrame `df` and columns like `"value"` and `"group"` in examples below.

Load Real Datasets (Iris, Wine, Diabetes)

```python
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
from sklearn.datasets import load_iris, load_wine, load_diabetes

iris = load_iris(as_frame=True).frame
wine = load_wine(as_frame=True).frame
diabetes = load_diabetes(as_frame=True).frame

# Convenience: readable group labels
iris["species"] = iris["target"].map({0:"setosa", 1:"versicolor", 2:"virginica"})
wine["class"] = wine["target"].map({0:"A", 1:"B", 2:"C"})
```

Histogram — Definition & When to Use

Definition. With bin edges $\{b_0, \ldots, b_K\}$, bar k shows

$$\text{count}_k = \#\{x_i \mid b_{k-1} < x_i \leq b_k\}, \quad \text{density}_k = \frac{\text{count}_k}{n\,(b_k - b_{k-1})}.$$

Use when: fast, intuitive view of shape; compare groups if you share bin edges or normalize.

Motivation

Start from *counts* for a single sample. When comparing groups or widths differ, move to *probability* or *density*. Add shared bins, cumulative view, and weights as needed.

Histogram → Start from Defaults (One-liner)

```
# START: quickest single-sample shape view
x = wine["alcohol"]
plt.figure()
plt.hist(x)  # default: ~10 equal-width bins, counts
plt.xlabel("alcohol"); plt.ylabel("count"); plt.title("Histogram: defaults")
plt.show()

# Seaborn equivalent (not executed):
sns.histplot(x=x, bins=10, stat="count", element="bars", kde=False)
plt.xlabel("alcohol"); plt.ylabel("count"); plt.title("Histogram: defaults")
plt.show()
```

Reading. Good for a quick, single-sample shape check (center, spread, rough modality).

Extend 1: Better Bins (Freedman–Diaconis)

```
x = wine["alcohol"]
plt.hist(x, bins="fd")            # data-adaptive; reduces oversmoothing/undersmoothing risk
plt.xlabel("alcohol"); plt.ylabel("count")

ax = sns.histplot(x=x, bins="fd", stat="count", element="bars", kde=False, discrete=False)

ax.set(xlabel="alcohol", ylabel="count", title="Histogram: bins='fd'")
plt.show()
```

Why. Default bin count can hide modes or add noise.

Extend 2: Probability vs Density (When & Why)

Probability bars sum to 1 (good for equal-width quick fairness).
Density integrates to 1 (robust to unequal bin widths, comparable to PDFs).

```
# Matplotlib
plt.hist(x, bins="fd", density=True)          # y = density (area = 1)

# Seaborn
sns.histplot(x=x, stat="probability")         # bar heights sum to 1
sns.histplot(x=x, stat="density")             # area under bars = 1
```

Use case. Comparing *shapes* across groups or overlaying theoretical densities.

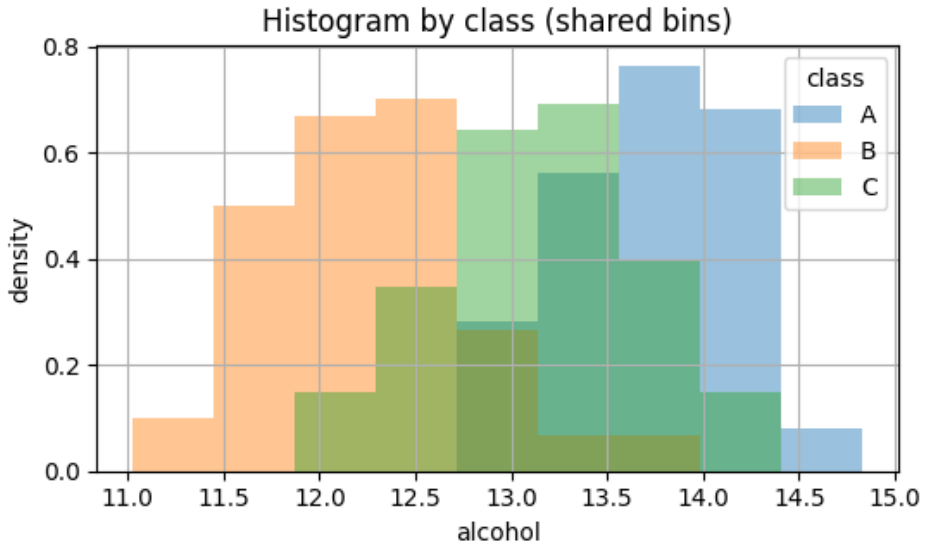
Extend 3: Fair Group Comparison — Concept & Visual

Concept (before code).

- *Shared bins:* Compute one set of bin edges $B = \{b_0, \ldots, b_K\}$ from the *pooled data* and apply the **same** edges to all groups.
- *Density normalization:* For group g and bin k,

$$\text{density}_{g,k} = \frac{\#\{x_i \in g : \; b_{k-1} < x_i \leq b_k\}}{n_g \, (b_k - b_{k-1})},$$

  so each group's histogram integrates to 1. This enables fair *shape* comparison even when sample sizes differ.
- **Why:** Different bins or raw *counts* can confound true shape differences with binning and sample-size artifacts.



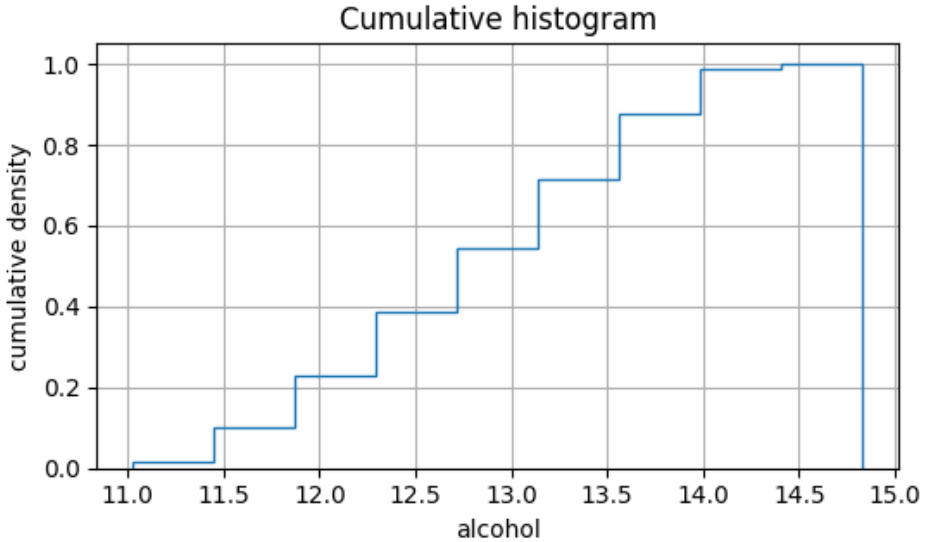
Extend 3: Shared Bins + Normalization — Code & Reading

```python
# Shared bins + density normalization
xcol = "alcohol"

# 1) Compute common bin edges from pooled data
bins = np.histogram_bin_edges(wine[xcol], bins="fd")

# 2) Apply same bins to all groups + density=True
for g, sub in wine.groupby("class"):
    plt.hist(sub[xcol], bins=bins,
             density=True,   # area = 1
             alpha=0.45, label=g)

plt.xlabel("alcohol")
plt.ylabel("density")
plt.legend(title="class")
plt.title("Histogram by class (shared bins, density)")
plt.show()

# Seaborn idea (reference):
sns.histplot(data=wine, x=xcol, hue="class",
              bins=bins, stat="density", common_bins=True)
```

- Darker overlaps (blended colors) show common mass; a group's bulge in a range marks its relative dominance there.
- Overlapping regions (darker where colors blend) show common mass; bulges unique to a group mark relative dominance on that range.

Extend 4: Cumulative Histogram (Bridge to ECDF)

```
plt.hist(x, bins="fd", density=True, cumulative=True, histtype="
    step")
plt.xlabel("alcohol"); plt.ylabel("cumulative density")
```



When needed. Quantiles and stochastic dominance; motivates ECDF for bin-free cumulative view.

ECDF — Bin-free Cumulative View

Empirical CDF

$$\hat{F}(x) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{x_i \le x\}, \qquad 0 \le \hat{F}(x) \le 1.$$

Why ECDF?

- **Bin-free:** no bin width or edges to choose; no histogram artifacts.
- **Robust comparison:** directly compare samples with *unequal* n.
- **Quantiles at a glance:** read medians/percentiles where $\hat{F}(x) = 0.5, 0.9, \ldots$

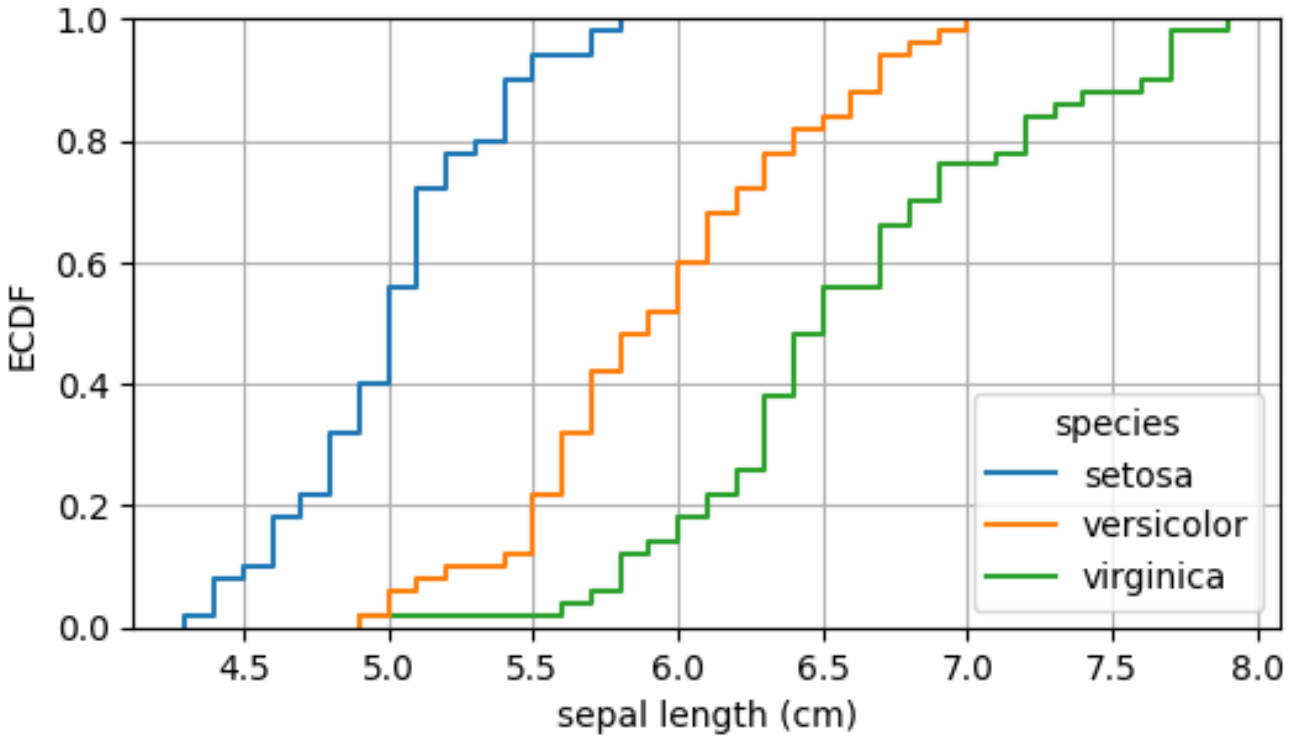
Properties & Tips

- Nondecreasing step function; jumps of size $1/n$ at the ordered data.
- Ties produce larger jumps (multiple points at the same x).
- Overlay multiple ECDFs to compare distributions; vertical gaps indicate stochastic dominance.
- Use the maximum vertical gap (KS distance) as a simple goodness-of-fit / two-sample diagnostic.

ECDF → Start (Default) and Groups

```
# Single sample
sns.ecdfplot(data=iris, x="sepal length (cm)")

# Group comparison (unequal n safe)
sns.ecdfplot(data=iris, x="sepal length (cm)", hue="species")
plt.ylabel("ECDF")
```

Reading. At x, height is fraction $\leq x$; right-shifted curve implies larger values.

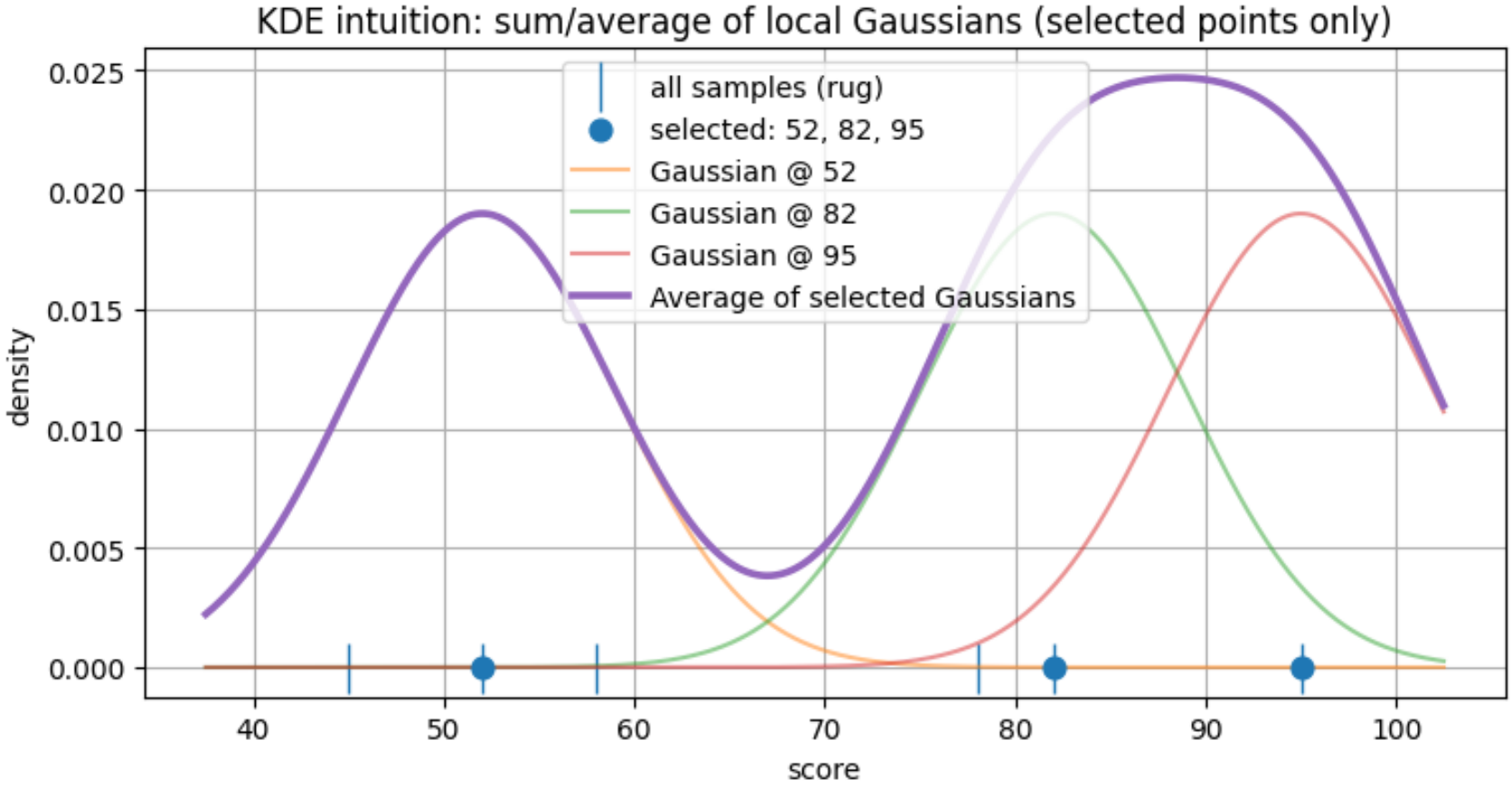


The Challenge: Estimating a Population Distribution

- **Problem.** From a small sample, estimate the *population* distribution.
- **Sample (n=6).** 45, 52, 58, 78, 82, 95
- **Intuition.** The sample hints at groups (e.g., ≈ 50, ≈ 80, a high achiever near 95).
- **KDE's role.** Turn this intuition into a smooth, quantitative estimate of the underlying density.

Fundamental Insight: Local Evidence

- **Local information.** Each observed x_i carries information about its *neighborhood*, not only x_i itself.
- **Kernel idea.** Place a bell-shaped probability curve (*kernel*) at each x_i to encode our confidence about nearby values.
- Each bell curve contributes **local evidence** for scores near x_i; farther away $\Rightarrow$ smaller contribution .



KDE Mechanism: Averaging Predictions

How it works

1. Center a kernel at each observation x_i.
2. For a query x, read each kernel's value at x (its prediction).
3. Average those predictions over all n points.

Estimator (with normalization).

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right), \qquad \int \hat{f}_h(x)\, dx = 1.$$

- K: kernel function (often Gaussian) .
- h: bandwidth controls the width of each kernel (smoothness knob) .

Critical Parameter: Bandwidth h

Too small ($h \downarrow$): *undersmooth*

- Jagged, noisy estimate.
- Many spurious peaks that may be artifacts.

Too large ($h \uparrow$): *oversmooth*

- Blurry, flat estimate.
- Misses genuine modes/structure.

Choosing h. Seek the *sweet spot* via cross-validation or plug-in rules.

In Seaborn. A data-driven base bandwidth is chosen; scale it with `bw_adjust`:

$$h_{\text{used}} = \texttt{bw_adjust} \times h_{\text{base}}.$$

Smaller `bw_adjust` $\Rightarrow$ more detail; larger $\Rightarrow$ smoother.

Practice: When to Use, Boundaries, and Groups

- **When to use.** Reveal modes and tail behavior beyond histogram binning; compare *shapes* across groups.
- **Boundaries.** For bounded data (e.g., $x \geq 0$), reduce boundary bias via *reflection* or *transformations* (e.g., log).
- **Groups.** For group comparisons, use consistent settings (common axis limits; if normalizing by density, compare shapes rather than counts).
- **Nonparametric power.** KDE adapts to whatever shape your data exhibits (unimodal, multimodal) without rigid assumptions.

Robust Summaries: Median, IQR, MAD

Median is the 50th percentile. **IQR** $= Q_3 - Q_1$. **MAD** $= \mathrm{median}(|x_i - \mathrm{median}(x)|)$.

Motivation

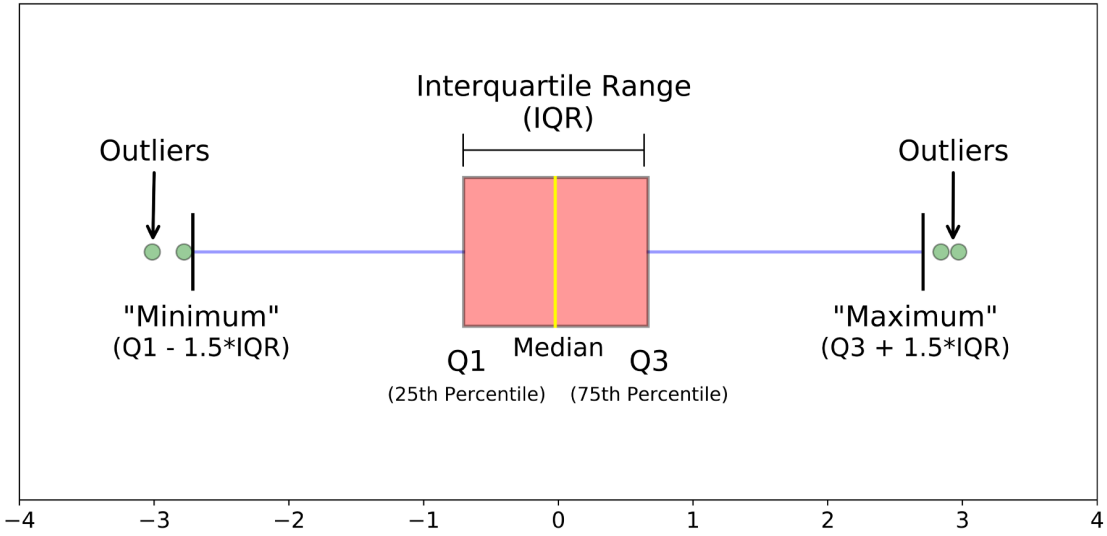
Summarize many groups compactly; add raw points for small n; prefer boxen to expose tails at large n.

Box vs Violin vs Boxen — What Are They For?

- **Box (Tukey).** Robust summary (median, IQR, whiskers, outliers). Fast to read; hides shape inside quartiles.
- **Violin.** Box *plus* KDE shape. Reveals multimodality, skewness, and tails; bandwidth-sensitive.
- **Boxen (Letter-Value).** Deeper quantiles beyond quartiles; built for large n to visualize tails reliably.

One-line takeaway: *The biggest difference between Box and Violin is the use of* ***KDE*** (Violin uses KDE; Box does not).

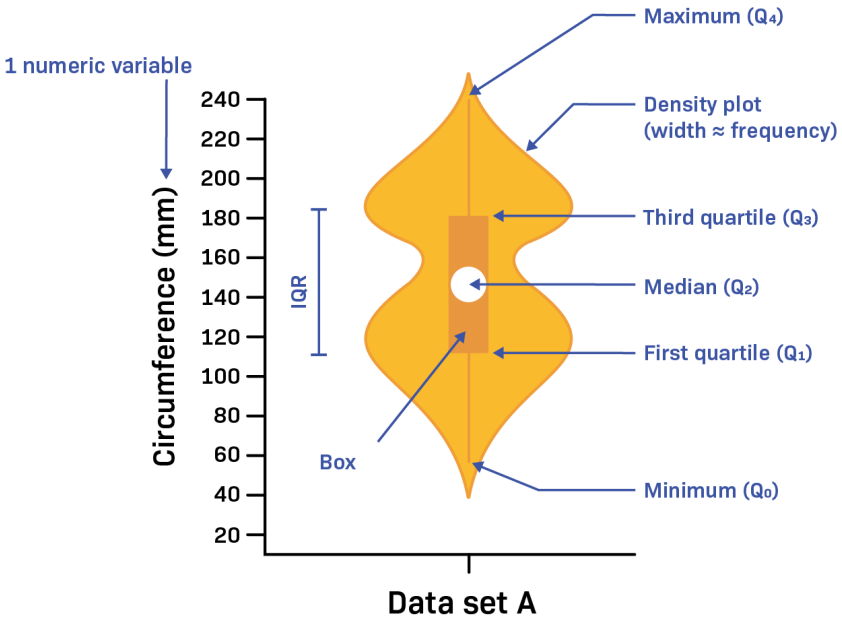
Box Plot: Anatomy & Defaults



- **Median line** inside the box; **box** spans Q1–Q3 (IQR).
- **Whiskers** typically extend to the most extreme points within 1.5×IQR from Q1/Q3.
- **Outliers** appear beyond whiskers (`showfliers=True` by default in seaborn).
- **Strengths.** Robust; compact; excellent for comparing medians and spreads.
- **Limits.** Cannot reveal multimodality or detailed shape within quartiles.

Seaborn knobs. `showfliers`, `whis` (e.g., `whis=(5,95)`), `notch=True`.

Violin Plot: Anatomy & Defaults



- **KDE envelope** mirrored about the center; width $\propto$ estimated density.
- **Inner marks** via `inner`: `"quartile"`, `"box"`, `"point"`, or `None`.
- **Bandwidth** matters: small $\Rightarrow$ noisy; large $\Rightarrow$ over-smooth.
- **Strengths.** Shows skew, tails, and multimodality; intuitive shape cue.
- **Limits.** Sensitive to bandwidth; small n can produce misleading bumps.

Key Distinction

Violin = Box + KDE shape. The biggest difference from Box is the use of **KDE**, where *width encodes density*.
Box shows only quantile summaries (median, IQR, whiskers, outliers).

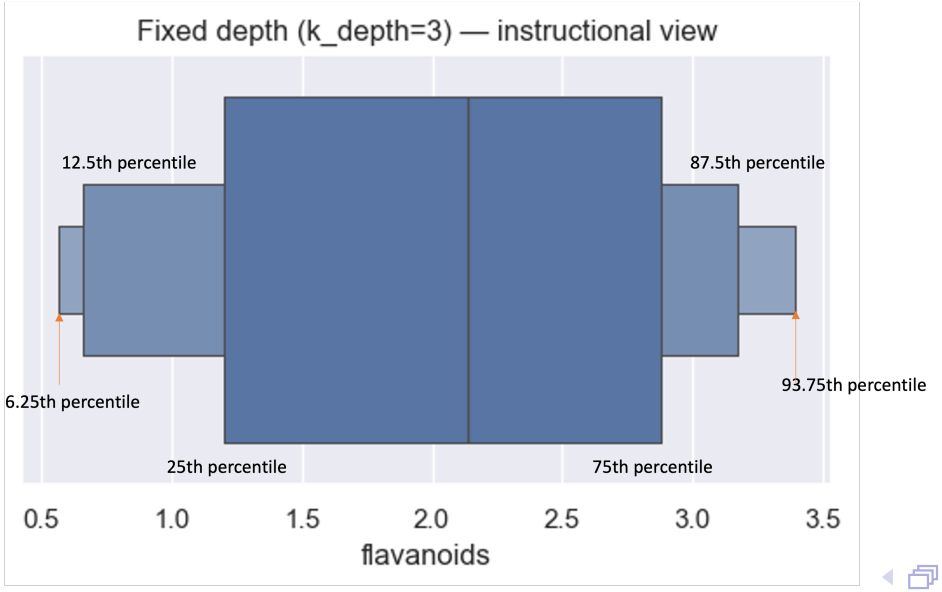
Boxen (Letter-Value) Plot — Definition

displays a *stack of nested boxes* built: the median and progressively deeper, symmetric quantile pairs that halve the tail mass at each step.

Construction (population view). Let $p_j = 2^{-j}$ for $j = 1, 2, \ldots, k$ (depth). At depth j, draw a box from $Q_{p_j/2}$ to $Q_{1-p_j/2}$:

$$\text{box}_j \;=\; \left[\, Q_{p_j/2},\; Q_{1-p_j/2} \,\right] \quad \Rightarrow \quad \text{central mass } = 1 - p_j.$$

j=1 gives the IQR box (25%–75%), j=2 gives 12.5%–87.5%, then 6.25%–93.75%, ...up to depth k. Boxes are drawn with decreasing width/thickness so deeper boxes are visually emphasized. A line marks the median.



Boxen Plot — What Seaborn Draws & How to Read It

Key Seaborn `boxenplot` options.

- `k_depth`: how many letter-value levels to draw. `"tukey"` (classical stopping, default), `"proportion"` (until a minimum tail proportion), `"trustworthy"` (as deep as reliable for sample size), or an integer (fixed depth).
- `scale`: how horizontal box widths shrink across depths (`"exponential"`, `"linear"`, `"area"`, ...).
- `outlier_prop`: tail proportion used to flag outliers (also used by `"proportion"` depth).
- `showfliers`: whether to plot outlier points.
- Others: `width`, `dodge` (when using `hue`), `linewidth`.

Boxen Plot — Seaborn Code (Updated API)

```python
# Example 1) Single distribution summary
ax = sns.boxenplot(
    data=wine, x="flavanoids",
    k_depth="tukey",            # automatic depth (default)
    width_method="exponential",# was 'scale'; use width_method in >=0.13
    outlier_prop=0.007,
    showfliers=True,
    linewidth=1.2,
)
ax.set_xlabel("flavanoids")
ax.set_title("Boxen (letter-value) summary")
plt.show()

# Example 2) Group comparison (with hue)
ax = sns.boxenplot(
    data=wine, x="class", y="flavanoids", hue="class",
    dodge=True, width=0.8,
    k_depth="trustworthy",      # as deep as reliable for n
    width_method="linear",      # (renamed from 'scale')
    outlier_prop=0.01,
    showfliers=True,
    linewidth=1.1,
)
ax.set_xlabel("class"); ax.set_ylabel("flavanoids")
ax.set_title("Boxen by class (deeper tail summaries)")

plt.show()

# Example 3) Fixed depth (instructional)
ax = sns.boxenplot(
    data=wine, x="flavanoids",
    k_depth=3,                  # draw j = 1..3 levels:
                                # [25  75  ], [12.5  87  .5], [6.25  93  .75]
    showfliers=False,
)
ax.set_title("Fixed depth (k_depth=3)     instructional view")
plt.show()
```

When to Use Which?

One-line takeaway: *The biggest difference between Box and Violin is KDE usage.* Violin emphasizes **shape via KDE**; Box emphasizes **robust summaries**.

- **Small n (< 50).** Prefer **Box** + raw points (`stripplot`/`swarmplot`) to avoid over-reading KDE bumps.
- **Shape matters.** Use **Violin**; tune `bw_adjust` and set `cut=0` to avoid beyond-range tails.
- **Large n.** Consider **Boxen** for deeper quantiles and tail comparison.
- **Fair comparison.** Keep axes consistent; avoid mixing `scale` modes that distort side-by-side interpretation.

Box vs Violin vs Boxen (Defaults Preserved)

```python
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(12, 3))

sns.boxplot(data=wine, x="class", y="alcohol", ax=axes[0])
axes[0].set_title("Box")

sns.violinplot(data=wine, x="class", y="alcohol",
               inner="quartile", cut=0, ax=axes[1])
axes[1].set_title("Violin")

sns.boxenplot(data=wine, x="class", y="alcohol", ax=axes[2])
axes[2].set_title("Boxen")

plt.tight_layout()
```

Guidance. Small n: box + raw points. Large n: boxen for deeper quantiles.

Pitfalls & Best Practices

- **Overplotting.** Add jittered points (`stripplot`) or use `swarmplot`; lower `alpha`.
- **Bandwidth sensitivity (Violin).** Tune `bw_adjust`; set `cut=0` to avoid spurious beyond-range tails.
- **Outliers (Box).** Choose `whis` and `showfliers` deliberately; document the rule.
- **Scaling.** Keep limits consistent across panels; avoid mixing `scale` modes in violins.
- **Explain the summary.** Label median, quartiles, whiskers, and what outliers represent to prevent misreads.

Rug & Strip — Show the Atoms

Rug. One tick per observation along an axis. **Strip.** A 1D scatter with small, random jitter to reveal overlap/ties.

Motivation

Aggregates (hist/KDE/box/violin) hide microstructure. Overlaying raw points (rug/strip) helps:

- **Sanity checks.** Does the KDE or box/violin reflect the actual locations of observations?
- **Small to medium n.** A quick read of clumps, gaps, and outliers.
- **Ties/discreteness.** Jitter reveals stacked/identical values that a plain scatter would overplot.

Interpretation

- **Density vs atoms.** Rug/strip show where points *are*; KDE/violin show a smoothed *tendency*.
- **Jitter is visual only.** It breaks overlap but does not change the underlying data; never quantify from jittered positions.

Design Choices That Matter

Rug.

- `height` (axis fraction): make ticks short so they do not dominate ($\approx$ 0.02–0.05).
- `lw`/`alpha`: thin and semi-transparent to avoid dark bands at large n.
- Orientation: horizontal via `x=` or vertical via `y=`.

Strip.

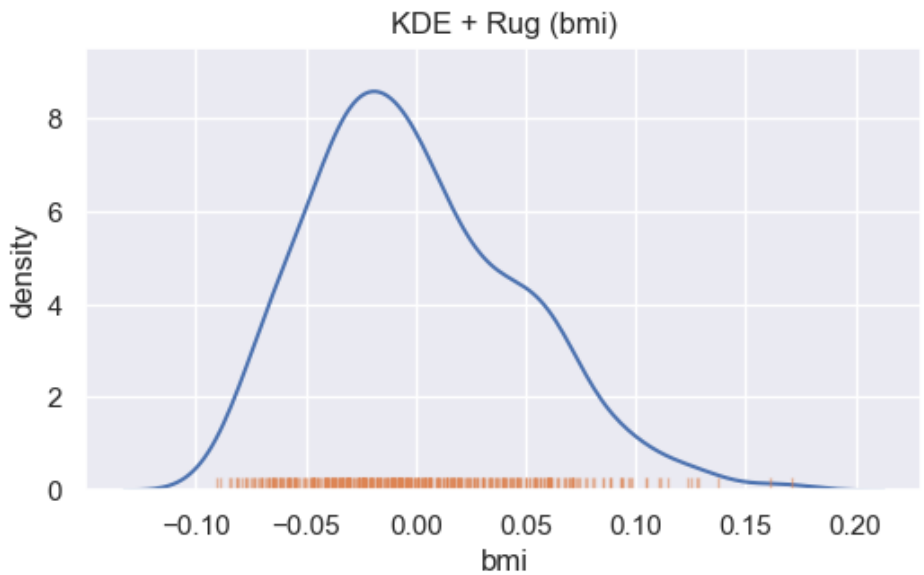
- `jitter` (float or `True`): just enough to resolve overlaps; too much looks like noise.
- `alpha`, `size`: lower alpha and smaller markers as n grows.
- `dodge` with `hue`: separate categories within each x level for fairer overlap.

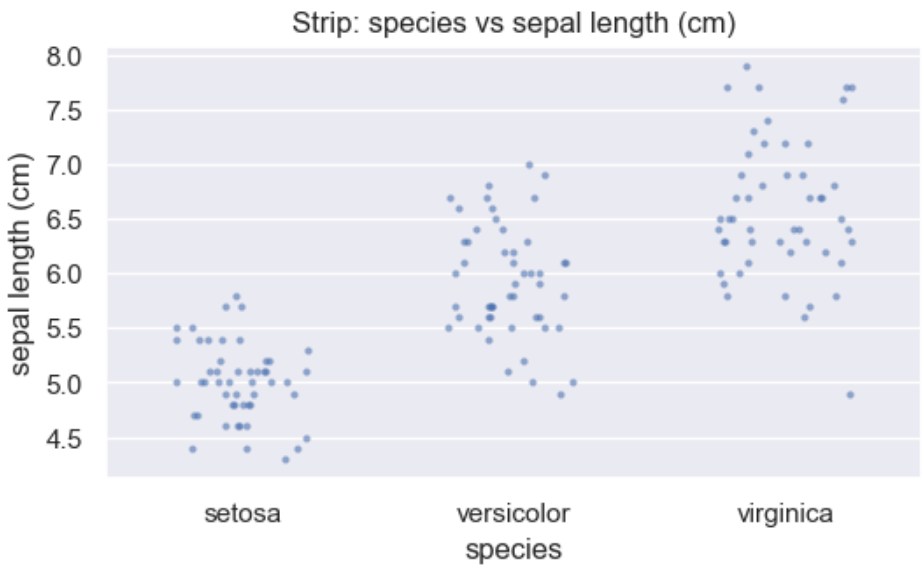
When n is large.

- Downsample; or use thin/transparent rug; or switch to `swarmplot` for moderate n (automatic non-overlap).
- Pair with aggregates (hist/KDE/ECDF) and keep atoms subtle.

Rug/Strip: Canonical Overlays (Seaborn)

```python
# 1) Rug + KDE (Diabetes: bmi)
fig, ax = plt.subplots()
sns.kdeplot(data=diabetes, x="bmi", ax=ax, bw_adjust=1.0)
sns.rugplot(data=diabetes, x="bmi", ax=ax, height=0.03, lw=0.6, alpha=0.5)
ax.set_title("KDE + Rug (bmi)"); ax.set_ylabel("density")

# 2) Strip by group (Iris species vs sepal length)
fig, ax = plt.subplots()
sns.stripplot(
    data=iris, x="species", y="sepal length (cm)",
    jitter=0.25, alpha=0.6, size=3, ax=ax
)
ax.set_title("Strip: species vs sepal length (cm)")
```





Q–Q Plot — Normality & Tails

Definition. For a sample $x_{1:n}$ with order statistics $x_{(i)}$ and probabilities $p_i = \frac{i-0.5}{n}$, the Q–Q plot against a reference CDF F places points

$$\left(F^{-1}(p_i),\ x_{(i)}\right), \quad i = 1, \ldots, n.$$

If the data follow F up to location/scale, points lie approximately on a straight line.

Motivation

- **Model checks.** Validate assumptions (e.g., Normal residuals in linear models).
- **Shape diagnostics.** Curvature reveals skew; tail divergence reveals heavy/light tails.
- **Outliers.** Strong end deviations signal unusual extremes.

Reading the line. With z–scored data vs Normal: slope ≈ 1, intercept ≈ 0 suggests a good fit; systematic curvature indicates misfit.

Q–Q Variants & Good Practices

Variants.

- Compare to different families: `dist="t"`, `"expon"`, `"chi2"` (choose parameters).
- Apply to **model residuals** to check distributional assumptions.

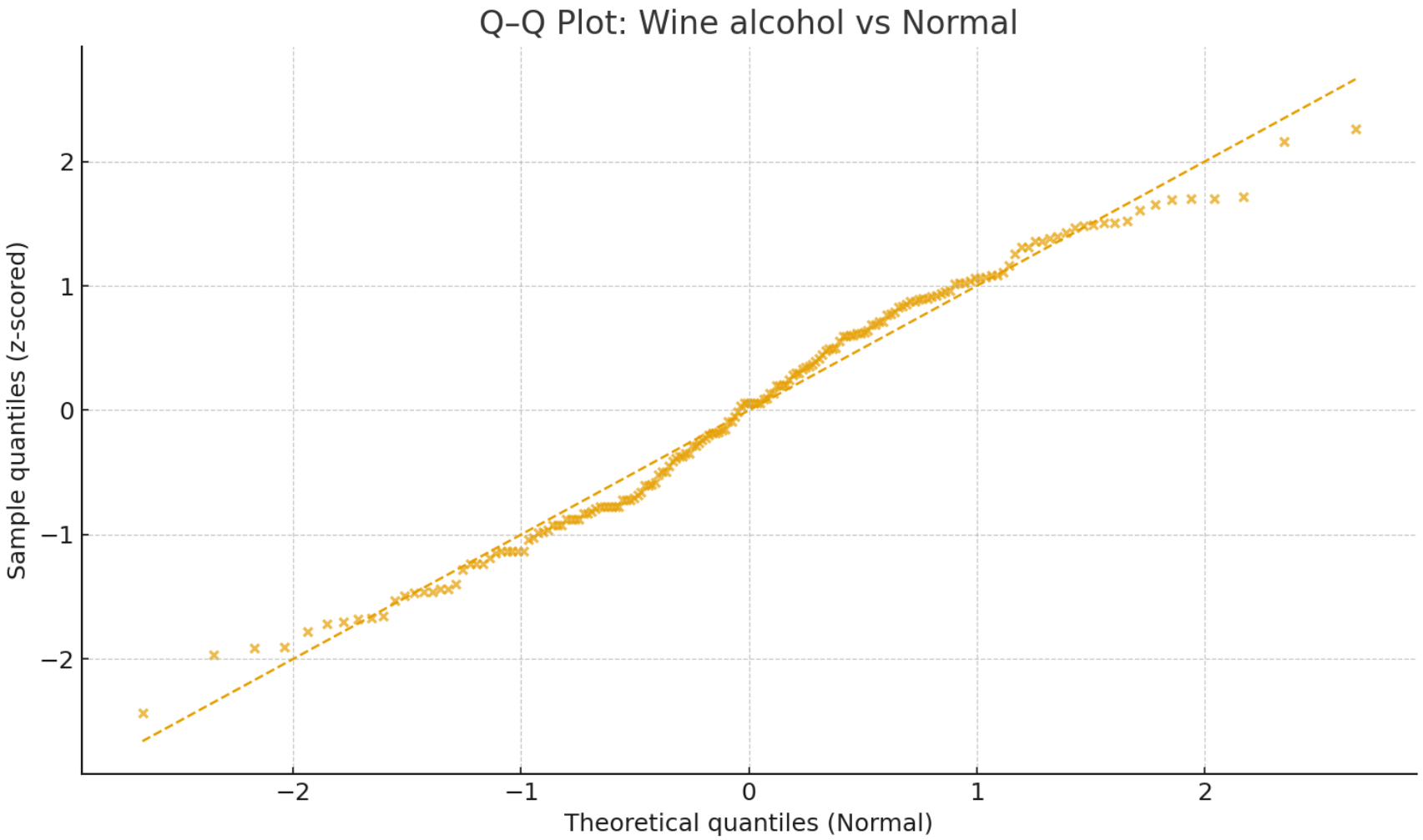
Good practices.

- **Standardize** before Normal Q–Q so slope/intercept are interpretable.
- For small n, modest deviations are common; annotate n and consider bands if needed.
- If robustness matters, also inspect median/MAD standardized values.

Q–Q (Wine: z–scored alcohol vs Normal) — Code

```python
import numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import load_wine
import scipy.stats as st

wine = load_wine(as_frame=True)
x = wine.frame["alcohol"].to_numpy()
x = (x - x.mean()) / x.std(ddof=0)          # z-score

# Theoretical (Normal) vs sample quantiles
osm, osr = st.probplot(x, dist="norm", fit=False)

plt.figure()
plt.scatter(osm, osr, s=18, alpha=0.7)
mn, mx = np.min(osm), np.max(osm)
plt.plot([mn, mx], [mn, mx], linestyle="--", linewidth=1.2)
plt.xlabel("Theoretical quantiles (Normal)")
plt.ylabel("Sample quantiles (z-scored)")
plt.title("Q--Q Plot: Wine alcohol vs Normal")
plt.tight_layout()
plt.savefig("qq_wine_alcohol.png", dpi=150)
```

Q–Q (Wine: alcohol) — Executed Result



Interpretation. The central region aligns closely with the line (near-Normal middle mass); a slight upward deviation on the right tail suggests somewhat heavier right tail or a few large values.

Putting It Together (Reading Strategy)

1. Start with **histogram (counts)** for a fast anchor.
2. For group comparison or unequal n: **probability/density** with **shared bins**.
3. Add **ECDF** for bin-free quantiles and stochastic dominance.
4. Add **KDE** for smooth modes; sanity-check with **rug/strip** when n is small.
5. Use **Box/Boxen/Violin** to summarize/compare groups.
6. Diagnose with **Q–Q**